

CH06 Synchronization

— race condition

— critical section problem

entry $\rightarrow$ exit $\rightarrow$ remainder
└─ critical

3가지 조건 { 상호배제 mutual exclusion
진행 progress
한정된 대기 bounded wait

2가지 접근법 { 선점 preemptive
비선점 non preemptive (못 찾아냄)

— peterson solution (2개만 제한)

int turn $\leftarrow$ 누구차례
bool flag[2] $\leftarrow$ 들어가도 준비했는지

— test and set solution



항상 원시값을 반환, 항상 false로 설정.

false면 값을 읽고, true면 바꿈. $\rightarrow$ 나중엔 false로 바꿔주기

- Compare and swap solution

(value, expected, new_value)

value == expected 이면 new_value로 교체 후 반환은 원래 value.

- mutex lock

acquire()

release()

- Semaphore (정수 값)

wait() -1 (0보다 작으면 wait)

signal() +1

↓ No busy waiting

waiting queue

└ value
 다음 pointer

block 대기(큐에 넣기)

wakeup ready queue에 넣기

wait() 하면 block, signal()하면 wakeup.

- Bounded buffer problem (producer - consumer)

initiator

┌ mutex = 1
 full = 0
 empty = n

Producer

wait(empty) wait(mutex)

인식

signal(mutex) signal(full)

Consumer

wait(full) wait(mutex)

인식

signal(mutex) signal(empty)

— readers - writers problem

— Dining Philosophers problem